

 DESARROLLO ANDROID

APIs: Fundamentos y Desarrollo con Android y Kotlin

Guía completa para construir aplicaciones conectadas con APIs RESTful



Android



Kotlin



APIs REST

Contenido

01

Introducción a las APIs

Conceptos fundamentales y arquitectura RESTful

02

Métodos HTTP y CRUD

GET, POST, PUT, DELETE y operaciones sobre recursos

03

JSON y Comunicación

Formato de datos y Kotlin Data Classes

04

Códigos de Estado HTTP

2xx, 4xx, 5xx y manejo de respuestas

05

Autenticación y Seguridad

JWT, OAuth y protección de APIs

06

Retrofit en Android

Cliente HTTP type-safe para Kotlin

07

Kotlin Coroutines

Programación asíncrona simplificada

08

Arquitectura MVVM

Separación de responsabilidades

09

Proyecto Completo

Aplicación de Lista de Usuarios con todas las capas

Introducción a las APIs

Conceptos fundamentales, arquitectura RESTful y anatomía de
peticiones HTTP



¿Qué es una API?

Definición

API (Application Programming Interface) es un conjunto de reglas y protocolos que permite que diferentes aplicaciones se comuniquen entre sí.

Actúa como un **intermediario** que recibe solicitudes, las procesa y devuelve respuestas.

Analogía: El Mesero

Imagina un restaurante: **Tú (Cliente)** ↔ **Mesero (API)** ↔ **Cocina (Servidor)**

- ✓ Tú pides el menú (GET request)
- ✓ El mesero lleva tu orden a la cocina
- ✓ La cocina prepara y el mesero trae tu comida (Response)

Tipos de APIs



APIs Privadas

Uso interno dentro de una organización. No expuestas públicamente.



APIs Partner

Compartidas con socios específicos bajo acuerdos comerciales.



APIs Públicas

Abiertas a desarrolladores externos. APIs de Google, Twitter, etc.

APIs Famosas



Google Maps API



Twitter API



Stripe API



Spotify API



Arquitectura RESTful

¿Qué es REST?

REST (Representational State Transfer) es un estilo arquitectónico para diseñar sistemas distribuidos. Fue creado por Roy Fielding en 2000.

Una API es **RESTful** cuando sigue los principios REST.

Principios REST

1 Stateless (Sin Estado)

Cada petición es independiente. El servidor no guarda estado entre peticiones.

2 Recursos como Sustantivos

Las URLs representan recursos (/users, /products), no acciones.

3 Uso de HTTP Methods

GET, POST, PUT, DELETE para operaciones CRUD.

4 Interfaz Uniforme

Mismo formato de respuesta para todos los recursos (JSON).

Anatomía de una Petición HTTP

1. URL / Endpoint

La dirección del recurso. Ejemplo: `https://api.ejemplo.com/v1/users/123`

2. Método HTTP

La acción a realizar: GET, POST, PUT, DELETE, PATCH

3. Headers

Metadatos: Content-Type, Authorization, Accept

4. Body (opcional)

Datos enviados en POST/PUT. Generalmente en formato JSON.

Anatomía de una Respuesta

200

Status Code

JSON

Headers

}

Body

Métodos HTTP y Operaciones CRUD

Operaciones fundamentales sobre recursos: Create, Read,
Update, Delete



Los 4 Métodos HTTP Principales



GET

Read (Leer)

Recupera datos de un recurso. No modifica el servidor.

- ✓ Idempotente: Sí
- ✓ Seguro: Sí
- ✓ Cacheable: Sí

GET /users/123



POST

Create (Crear)

Crea un nuevo recurso. Envía datos en el body.

- ✗ Idempotente: No
- ✗ Seguro: No
- ✓ Cacheable: Condicional

POST /users



PUT

Update (Actualizar)

Reemplaza completamente un recurso existente.

- ✓ Idempotente: Sí
- ✗ Seguro: No
- ✗ Cacheable: No

PUT /users/123



DELETE

Delete (Eliminar)

Elimina un recurso del servidor.

- ✓ Idempotente: Sí
- ✗ Seguro: No
- ✗ Cacheable: No

DELETE /users/123



PUT, PATCH y DELETE en Detalle

PUT vs PATCH

PUT - Actualización Completa

Reemplaza **todo** el recurso. Debes enviar todos los campos.

```
// PUT /users/123
{
  "id": 123,
  "name": "Juan",
  "email": "juan@mail.com",
  "phone": "555-1234"
}
```

PATCH - Actualización Parcial

Actualiza **solo** los campos enviados. Más eficiente.

```
// PATCH /users/123
{
  "email": "nuevo@mail.com"
}
```

¿Cuándo usar cada uno?

- **PUT**: Cuando actualizas todo el objeto
- **PATCH**: Cuando actualizas un campo específico

DELETE

Elimina un recurso permanentemente. Generalmente no requiere body.

```
// DELETE /users/123
```

```
// Respuesta típica: 204 No Content
```

204

No Content

404

Not Found

Ejemplos Prácticos

GET /api/products

Obtener todos los productos

POST /api/orders

Crear nueva orden

PUT /api/users/123

Actualizar usuario completo

DELETE /api/cart/items/5

Eliminar item del carrito

03 CAPÍTULO

JSON y Comunicación de Datos

Formato de intercambio estándar y mapeo con Kotlin



JSON y Kotlin Data Classes

¿Qué es JSON?

JSON (JavaScript Object Notation) es un formato ligero de intercambio de datos. Es fácil de leer y escribir para humanos, y fácil de parsear y generar para máquinas.

- ✓ Ligero
- ✓ Legible
- ✓ Language-agnostic
- ✓ Estándar en APIs

Estructura de JSON

Objetos

```
{ "clave": "valor" }
```

Arrays

```
[ "item1", "item2" ]
```

Tipos de Datos

string, number, boolean, null, object, array

Kotlin Data Classes

Las **data classes** de Kotlin son perfectas para mapear respuestas JSON. Generan automáticamente `equals()`, `hashCode()`, `toString()`, `copy()` y `componentN()`.

```
data class User(  
    val id: Int,  
    val name: String,  
    val email: String,  
    val avatar: String?  
)
```

Mapeo JSON → Kotlin

```
// JSON de la API  
{  
    "id": 1,  
    "name": "Ana García",  
    "email": "ana@mail.com",  
    "avatar": "https://..."  
}
```

 **Tip:** Usa `@SerializedName("user_name")` si el nombre JSON difiere del campo Kotlin.

Códigos de Estado HTTP

Comunicación de resultados: éxito, errores del cliente y del servidor



Códigos 2xx: Éxito

Los códigos 2xx indican que la petición fue recibida, entendida y aceptada exitosamente. Son la respuesta más deseada en cualquier comunicación API.

200 OK

Éxito estándar

La petición fue exitosa. El significado depende del método HTTP usado.

- **GET:** Recurso encontrado y devuelto
- **POST:** Recurso creado exitosamente
- **PUT/DELETE:** Operación completada

201 Created

Recurso creado

La petición fue exitosa y resultó en la creación de un nuevo recurso.

```
// Respuesta típica
Status: 201 Created
Location: /users/123
```



Usar con POST cuando se crea un recurso nuevo

204 No Content

Sin contenido

La petición fue exitosa pero no hay contenido para devolver en la respuesta.

```
// Respuesta típica
Status: 204 No Content
// Body vacío
```



Ideal para DELETE o PUT exitosos

202

Accepted

Aceptado para procesar

204

No Content

Éxito sin body

206

Partial Content

Contenido parcial

200

OK

Éxito general



Códigos 4xx: Errores del Cliente

Los códigos 4xx indican que hubo un error por parte del cliente. La petición contiene sintaxis incorrecta o no puede ser procesada.

400

Bad Request

Solicitud incorrecta

El servidor no puede procesar la petición debido a un error del cliente (sintaxis JSON inválida, parámetros faltantes).

```
// Ejemplo: email inválido
{
  "error": "INVALID_EMAIL",
  "message": "Formato de email inválido"
}
```

401

Unauthorized

No autorizado

Se requiere autenticación. El cliente debe iniciar sesión primero.

```
// Falta token o token inválido
{
  "error": "UNAUTHORIZED",
  "message": "Token de acceso requerido"
}
```

403

Forbidden

Prohibido

El cliente está autenticado pero no tiene permisos para acceder al recurso.

401 vs 403:

- 401: "¿Quién eres?" (No autenticado)
- 403: "No puedes entrar" (Sin permisos)

404

Not Found

No encontrado

El recurso solicitado no existe en el servidor.

```
// GET /users/999 (usuario no existe)
{
  "error": "NOT_FOUND",
  "message": "Usuario con ID 999 no existe"
}
```

409

Conflict

Conflicto

La petición no pudo completarse debido a un conflicto con el estado actual del recurso.



Códigos 5xx: Errores del Servidor

Los códigos **5xx** indican que el servidor encontró un error al procesar una petición válida. Estos errores son responsabilidad del servidor, no del cliente.

500

Internal Server Error

Error interno del servidor

Error genérico cuando el servidor encuentra una condición inesperada que le impide completar la petición.

- ✖ Excepción no manejada
- ✖ Error de configuración
- ✖ Bug en el código

502

Bad Gateway

Puerta de enlace incorrecta

El servidor, actuando como puerta de enlace o proxy, recibió una respuesta inválida del servidor upstream.

- ✖ Servicio downstream caído
- ✖ Timeout de microservicio
- ✖ Proxy mal configurado

503

Service Unavailable

Servicio no disponible

El servidor no puede manejar la petición temporalmente (sobrecarga o mantenimiento).

Retry-After: 3600

Intentar nuevamente después de 1 hora

504

Gateway Timeout

Tiempo de espera agotado

El servidor no recibió una respuesta oportuna del servidor upstream.

- 🕒 Query de base de datos lenta
- 🕒 Servicio externo no responde
- 🕒 Procesamiento excesivo

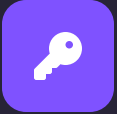
05 CAPÍTULO

Autenticación y Seguridad

Protegiendo el acceso a APIs: JWT, OAuth y mejores prácticas



Métodos de Autenticación



API Keys

Claves de API

Token simple enviado en cada petición. Fácil de implementar pero menos seguro.

```
// Header típico  
X-API-Key: abc123xyz789
```

- + Simple de implementar
- + Buen para servicios internos
- Menos seguro



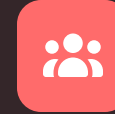
JWT

JSON Web Tokens

Token firmado que contiene información del usuario. Stateless y auto-contenido.

```
// Estructura JWT  
header.payload.signature
```

- + Stateless (sin sesión)
- + Contiene claims/info
- No revocable fácilmente



OAuth 2.0

Autorización delegada

Permite que aplicaciones de terceros accedan a recursos en nombre del usuario.

```
// Login con Google  
Authorization: Bearer token
```

- + Granular (scopes)
- + Revocable por usuario
- Complejo de implementar

Estructura de un JWT

HEADER

```
{ "alg": "HS256", "typ": "JWT" }
```

PAYLOAD

```
{ "userId": 123, "exp": 123456 }
```

SIGNATURE

```
HMACSHA256(...)
```

06 CAPÍTULO

Retrofit en Android

Cliente HTTP type-safe para Kotlin y Java



Introducción a Retrofit

¿Qué es Retrofit?

Retrofit es una librería de Square que convierte tu API HTTP en una interfaz Java/Kotlin. Es type-safe, lo que significa que los errores se detectan en tiempo de compilación.

- ✓ Type-safe
- ✓ Annotations
- ✓ Sinc/Async
- ✓ Converters

Dependencias Gradle

```
// build.gradle.kts (Module: app)
dependencies {
    // Retrofit
    implementation("com.squareup.retrofit2:retrofit:2.9.0")
    // Gson Converter
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")
    // Logging Interceptor (opcional)
    implementation("com.squareup.okhttp3:logging-interceptor:4.11.0")
}
```

Configuración Básica

```
object RetrofitInstance {
    private const val BASE_URL = "https://api.ejemplo.com/"
    val api: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(
                GsonConverterFactory.create()
            )
            .build()
            .create(ApiService::class.java)
    }
}
```

Ventajas sobre HttpURLConnection

- ★ **Type-safe:** Errores en compilación, no runtime
- ★ **Annotations:** Código más limpio y declarativo
- ★ **Converters:** JSON automático con Gson/Moshi
- ★ **Interceptors:** Logging, auth, headers fácilmente

</> Definiendo Endpoints con Retrofit

Anotaciones de Retrofit

@GET /users

Obtener lista de usuarios

@POST /users

Crear nuevo usuario

@PUT /users/{id}

Actualizar usuario completo

@DELETE /users/{id}

Eliminar usuario

Parámetros

@Path Variable en URL: /users/{id}

@Query Parámetro query: ?page=1

@Body Objeto en body JSON

@Header Header personalizado

Ejemplo Completo: UserApiService

```
interface UserApiService {  
    // GET - Obtener todos los usuarios  
    @GET("users")  
    suspend fun getUsers(): Response<List<User>>  
    // GET - Obtener usuario por ID  
    @GET("users/{id}")  
    suspend fun getUserById(  
        @Path("id") id: Int  
    ): Response<User>  
    // POST - Crear usuario  
    @POST("users")  
    suspend fun createUser(  
        @Body user: User  
    ): Response<User>  
    // PUT - Actualizar usuario  
    @PUT("users/{id}")  
    suspend fun updateUser(  
        @Path("id") id: Int,  
        @Body user: User  
    ): Response<User>  
    // DELETE - Eliminar usuario  
    @DELETE("users/{id}")  
    suspend fun deleteUser(  
        @Path("id") id: Int  
    ): Response<Unit>  
    // GET con query params  
    @GET("users")  
    suspend fun searchUsers(  
        @Query("name") name: String,  
        @Query("page") page: Int = 1  
    ): Response<List<User>>  
}
```

07 CAPÍTULO

Kotlin

Coroutines

Programación asíncrona simplificada en Android

Coroutines en Android

¿Qué son las Coroutines?

Coroutines son threads ligeros que permiten escribir código asíncrono de forma secuencial. Son la forma recomendada de manejar operaciones asíncronas en Kotlin.

- ✓ Lightweight
- ✓ Cancellable
- ✓ Secuencial
- ✓ Structured concurrency

Dispatchers

Dispatchers.Main

UI thread - actualizaciones de interfaz

Dispatchers.IO

Network, files, database - operaciones de I/O

Dispatchers.Default

CPU-intensive work - cálculos complejos

launch vs async

launch - Fire and Forget

```
val job = launch {  
    // Tarea en background  
    fetchData()  
}  
job.join() // Esperar
```

async - Devuelve resultado

```
val deferred = async {  
    fetchUser()  
}  
val user = deferred.await()
```

Scopes en Android

viewModelScope Vinculado al ciclo de vida del ViewModel

lifecycleScope Vinculado al ciclo de vida de Activity/Fragment



Integrando Coroutines con Retrofit

Llamadas API con Coroutines

Retrofit soporta coroutines nativamente usando suspend functions. Esto elimina la necesidad de Callbacks

a

```
class UserRepository {
    private val api = RetrofitInstance.api
    suspend fun getUsers(): Result<List<User>> {
        return try {
            val response = api.getUsers()
            if (response.isSuccessful) {
                Result.success(
                    response.body() ?: emptyList()
                )
            } else {
                Result.failure(
                    Exception("Error: ${response.code()}")
                )
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

Manejo de Errores

- ❗ **IOException:** Error de red, sin conexión
- ❗ **HttpException:** Error HTTP (4xx, 5xx)
- ❗ **JsonSyntaxException:** JSON mal formado
- ❗ **TimeoutException:** Tiempo agotado

ViewModel con Coroutines

```
class UserViewModel : ViewModel() {
    private val repository = UserRepository()
    private val _users = MutableStateFlow<List<User>>(emptyList())
    val users: StateFlow<List<User>> = _users
    fun loadUsers() {
        viewModelScope.launch {
            val result = withContext(Dispatchers.IO) {
                repository.getUsers()
            }
            result.fold(
                onSuccess = { _users.value = it },
                onFailure = { /* Manejar error */ }
            )
        }
    }
}
```

Operaciones Paralelas

```
suspend fun loadDashboard() =
    coroutineScope {
        val users = async { api.getUsers() }
        val orders = async { api.getOrders() }
        val stats = async { api.getStats() }
        Dashboard(
            users = users.await(),
            orders = orders.await(),
            stats = stats.await()
        )
    }
```

08 CAPÍTULO

Arquitectura MVVM

Separación de responsabilidades en Android



Patrón MVVM y Flujo Completo

¿Qué es MVVM?

MVVM (Model-View-ViewModel) es un patrón arquitectónico que separa la lógica de presentación de la lógica de negocio.

Model

Datos y lógica de negocio

View

UI - Activities, Fragments, Composables

ViewModel

Intermediario - Expone datos a la UI

Beneficios de MVVM

- ✓ **Testable:** Cada capa puede probarse independientemente
- ✓ **Maintainable:** Cambios en UI no afectan lógica
- ✓ **Scalable:** Fácil agregar nuevas features
- ✓ **Lifecycle-aware:** ViewModel sobrevive a rotaciones

Flujo Completo: API → UI



1. UI (View)

Usuario hace tap → Llama a ViewModel



2. ViewModel

Expone StateFlow → Llama a Repository



3. Repository

Abstrae fuente de datos → Llama a Retrofit



4. Retrofit

Hace petición HTTP → Espera respuesta



5. API Server

Procesa petición → Devuelve JSON



6. Response Flow

JSON → Retrofit → Repository → ViewModel → UI

09 CAPÍTULO

Proyecto Completo

Aplicación de Lista de Usuarios con todas las capas



Estructura del Proyecto

Organización de Paquetes

```
com.ejemplo.userapp/  
├── data/  
│   ├── model/ // Data classes  
│   ├── network/ // Retrofit, API Service  
│   └── repository/ // UserRepository  
├── ui/  
│   ├── screens/ // Composables  
│   ├── theme/ // Colors, Typography  
│   └── viewmodel/ // UserViewModel  
├── di/ // Dependency Injection  
└── utils/ // Helpers, Extensions
```

Capas de la Arquitectura

1

Presentation

UI, ViewModel, States

2

Domain

Repository interfaces, Use Cases

3

Data

Repository impl, API, Models

Dependencias Gradle

```
// build.gradle.kts (Module: app)  
dependencies {  
    // AndroidX Core  
    implementation("androidx.core:core-ktx:1.12.0")  
    implementation("androidx.lifecycle:lifecycle-runtime-ktx:2.7.0")  
    // Compose  
    implementation("androidx.compose.ui:ui:1.6.0")  
    implementation("androidx.compose.material3:material3:1.2.0")  
    implementation("androidx.activity:activity-compose:1.8.2")  
    // ViewModel  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.7.0")  
    // Retrofit  
    implementation("com.squareup.retrofit2:retrofit:2.9.0")  
    implementation("com.squareup.retrofit2:converter-gson:2.9.0")  
    // Coroutines  
    implementation("org.jetbrains.kotlin:kotlinx-coroutines-android:1.7.3")  
    // Coil (imágenes)  
    implementation("io.coil-kt:coil-compose:2.5.0")  
}
```



Paso 1-2: Modelo y Retrofit

Paso 1: Data Class User

```
data class User(  
    @SerializedName("id")  
    val id: Int,  
    @SerializedName("name")  
    val name: String,  
    @SerializedName("email")  
    val email: String,  
    @SerializedName("avatar")  
    val avatar: String? = null  
)
```

Paso 2: UserApiService

```
interface UserApiService {  
    @GET("users")  
    suspend fun getUsers(): Response<List<User>>  
    @GET("users/{id}")  
    suspend fun getUserById(  
        @Path("id") id: Int  
    ): Response<User>  
    @POST("users")  
    suspend fun createUser(  
        @Body user: User  
    ): Response<User>  
    @PUT("users/{id}")  
    suspend fun updateUser(  
        @Path("id") id: Int,  
        @Body user: User  
    ): Response<User>  
    @DELETE("users/{id}")  
    suspend fun deleteUser(  
        @Path("id") id: Int  
    ): Response<Unit>  
}
```

Configuración Retrofit

```
object RetrofitInstance {  
    private const val BASE_URL = "https://reqres.in/api/"  
    private val logging = HttpLoggingInterceptor().apply {  
        level = HttpLoggingInterceptor.Level.BODY  
    }  
    private val client = OkHttpClient.Builder()  
        .addInterceptor(logging)  
        .build()  
    val api: UserApiService by lazy {  
        Retrofit.Builder()  
            .baseUrl(BASE_URL)  
            .client(client)  
            .addConverterFactory(  
                GsonConverterFactory.create()  
            )  
            .build()  
            .create(UserApiService::class.java)  
    }  
}
```

API de Prueba: ReqRes



Usaremos **ReqRes**, una API REST pública para testing.



✓ Base URL: <https://reqres.in/api/>



✓ GET /users - Lista de usuarios

GET /users/{id} - Usuario específico

POST /users - Crear usuario

Paso 3-4: Repository y ViewModel

Paso 3: UserRepository

```
class UserRepository {
    private val api = RetrofitInstance.api
    suspend fun getUsers(): Result<List<User>> {
        return try {
            val response = api.getUsers()
            if (response.isSuccessful) {
                Result.success(
                    response.body() ?: emptyList()
                )
            } else {
                Result.failure(
                    Exception("Error: ${response.code()}")
                )
            }
        } catch (e: IOException) {
            Result.failure(Exception("Sin conexión"))
        } catch (e: Exception) {
            Result.failure(e)
        }
    }

    suspend fun createUser(user: User): Result<User> {
        return try {
            val response = api.createUser(user)
            if (response.isSuccessful) {
                Result.success(response.body()!!)
            } else {
                Result.failure(Exception("Error al crear"))
            }
        } catch (e: Exception) {
            Result.failure(e)
        }
    }
}
```

Paso 4: UserModel

```
class UserModel : ViewModel() {
    private val repository = UserRepository()
    sealed class UiState {
        object Loading : UiState()
        data class Success(val users: List<User>) : UiState()
        data class Error(val message: String) : UiState()
    }

    private val _uiState = MutableStateFlow<UiState>(UiState.Loading)
    val uiState: StateFlow<UiState> = _uiState
    fun loadUsers() {
        viewModelScope.launch {
            _uiState.value = UiState.Loading
            val result = withContext(Dispatchers.IO) {
                repository.getUsers()
            }
            result.fold(
                onSuccess = { _uiState.value = UiState.Success(it) },
                onFailure = { _uiState.value = UiState.Error(it.message ?: "Error") }
            )
        }
    }
    init { loadUsers() }
}
```

Estados UI con Sealed Class

Las **sealed classes** permiten representar estados de UI de forma type-safe y exhaustiva.

Loading Mostrar ProgressIndicator

Success Mostrar lista de usuarios

Error Mostrar mensaje de error



Paso 5-6: UI y Manejo de Errores

Paso 5: UserListScreen

```
@Composable
fun UserListScreen(
    viewModel: UserViewModel = viewModel()
) {
    val uiState by viewModel.uiState.collectAsState()
    when (val state = uiState) {
        is UserViewModel.UiState.Loading -> {
            Box(modifier = Modifier.fillMaxSize()) {
                CircularProgressIndicator(
                    modifier = Modifier.align(Alignment.Center)
                )
            }
        }
        is UserViewModel.UiState.Success -> {
            LazyColumn {
                items(state.users) { user ->
                    UserItem(user = user)
                }
            }
        }
        is UserViewModel.UiState.Error -> {
            Box(modifier = Modifier.fillMaxSize()) {
                Column(
                    modifier = Modifier.align(Alignment.Center),
                    horizontalAlignment = Alignment.CenterHorizontally
                ) {
                    Text("Error: ${state.message}")
                    Button(onClick = { viewModel.loadUsers() }) {
                        Text("Reintentar")
                    }
                }
            }
        }
    }
}
```

UserItem Composable

```
@Composable
fun UserItem(user: User) {
    Card(
        modifier = Modifier
            .fillMaxWidth()
            .padding(horizontal = 16.dp, vertical = 8.dp)
    ) {
        Row(
            modifier = Modifier.padding(16.dp),
            verticalAlignment = Alignment.CenterVertically
        ) {
            AsyncImage(
                model = user.avatar,
                contentDescription = null,
                modifier = Modifier
                    .size(50.dp)
                    .clip(CircleShape)
            )
            Spacer(modifier = Modifier.width(16.dp))
            Column {
                Text(
                    text = user.name,
                    style = MaterialTheme.typography.titleMedium
                )
                Text(
                    text = user.email,
                    style = MaterialTheme.typography.bodyMedium
                )
            }
        }
    }
}
```

Paso 6: Manejo de Errores

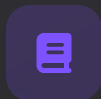
-  **Sin conexión:** Mensaje amigable + botón reintentar
-  **Timeout:** Indicar que el servidor está lento
-  **500 Error:** "Problema temporal, intenta más tarde"
-  **404 Not Found:** "Usuario no encontrado"



¡FELICIDADES!

¡Listo para Construir Apps!

Ahora tienes los conocimientos fundamentales para crear aplicaciones Android conectadas a APIs RESTful usando Kotlin, Retrofit y Coroutines.



Documentación

- Retrofit Guide
- Kotlin Coroutines
- Android Architecture



Próximos Pasos

- Paginación con Paging 3
- Caché offline con Room
- Testing de ViewModels



Mejores Prácticas

- Dependency Injection
- Error Handling robusto
- Loading states

Recuerda: **Practica**, **Experimenta**, **Construye**

Taller

Desarrollar , corregir y/o actualizar (de ser requerido) y de manera individual el taller propuesto en :

<https://keepcoding.io/blog/retrofit-en-kotlin-para-consumir-apis/>